

Multi-Particle Neural Operator Transformer (MPNOT)

1 Reference

The Multi-Particle Neural Operator Transformer (MPNOT) is introduced as a neural operator architecture that establishes a connection between Transformer attention mechanisms and multi-particle dynamical systems.

S. Liu, Y. Yu, C. Fan, T. Zhang, H. Liu, and X. Liu, “Multi-particle neural operator transformer for solving partial differential equations,” *Neural Networks*, vol. 202, 109040, 2026.

DOI: [10.1016/j.neunet.2026.109040](https://doi.org/10.1016/j.neunet.2026.109040).

2 Introduction

The Multi-Particle Neural Operator Transformer (MPNOT) is designed for learning nonlinear operators associated with partial differential equations.

The architecture is motivated by the interpretation of Transformer attention as an interacting particle system.

Consider a nonlinear PDE

$$\mathcal{F}(x, a, u) = f, \quad x \in \Omega, \quad (1)$$

with boundary condition

$$\mathcal{B}(u) = u_b, \quad x \in \partial\Omega, \quad (2)$$

and initial condition

$$\mathcal{I}(u) = u_0. \quad (3)$$

The objective of an operator-learning model is to learn the mapping

$$\mathcal{G} : (x, u_0, u_b, a, f) \longmapsto u. \quad (4)$$

MPNOT constructs this mapping using a Transformer architecture whose attention mechanism is interpreted through multi-particle dynamics.

3 Particle Representation

Let the input function be discretized at N spatial points,

$$\{x_i\}_{i=1}^N. \quad (5)$$

Each point is associated with a feature

$$h_i^0 = h(x_i). \quad (6)$$

The collection of all particles is

$$H^0 = [h_1^0, h_2^0, \dots, h_N^0]^T. \quad (7)$$

The neural operator learns the evolution

$$H^0 \longrightarrow H^1 \longrightarrow \dots \longrightarrow H^L. \quad (8)$$

The particle interpretation treats each token as a particle whose state evolves through interactions with the remaining particles.

4 Multi-Particle Dynamical System

Consider a system of N interacting particles with states

$$z_i(t) \in \mathbb{R}^d. \quad (9)$$

A general interacting-particle system can be written as

$$\frac{dz_i}{dt} = \sum_{j=1}^N K(z_i, z_j) z_j. \quad (10)$$

The discrete evolution over a time step Δt is

$$z_i^{n+1} = z_i^n + \Delta t \sum_{j=1}^N K(z_i^n, z_j^n) z_j^n. \quad (11)$$

This provides the conceptual connection between particle dynamics and attention-based neural networks.

5 Self-Attention

Given an input matrix

$$H \in \mathbb{R}^{N \times d}, \quad (12)$$

the query, key, and value matrices are

$$Q = HW_Q, \quad (13)$$

$$K = HW_K, \quad (14)$$

and

$$V = HW_V. \quad (15)$$

The attention score between particles i and j is

$$s_{ij} = \frac{Q_i K_j^T}{\sqrt{d_k}}. \quad (16)$$

The normalized interaction coefficient is

$$A_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^N \exp(s_{ik})}. \quad (17)$$

In matrix form,

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right). \quad (18)$$

The standard self-attention operation is

$$\text{Attention}(Q, K, V) = AV. \quad (19)$$

6 Multi-Particle Interpretation

For particle i ,

$$(AV)_i = \sum_{j=1}^N A_{ij} V_j. \quad (20)$$

Therefore the attention mechanism can be interpreted as a weighted interaction between particles.

The corresponding particle interaction equation is

$$h_i^{n+1} = h_i^n + \Delta t \sum_{j=1}^N A_{ij}^n V_j^n. \quad (21)$$

This resembles the forward-Euler discretization of

$$\frac{dh_i}{dt} = \sum_{j=1}^N A_{ij}(t) V_j(t). \quad (22)$$

The attention matrix therefore acts as an interaction kernel between particles.

7 Multi-Particle Attention Layer

MPNOT introduces a multi-particle attention mechanism designed to capture interactions among particle states.

Let

$$H^l \in \mathbb{R}^{N \times d} \quad (23)$$

denote the representation at layer l .

The query, key, and value transformations are

$$Q^l = H^l W_Q^l, \quad (24)$$

$$K^l = H^l W_K^l, \quad (25)$$

and

$$V^l = H^l W_V^l. \quad (26)$$

The pairwise particle interaction is

$$e_{ij}^l = \frac{Q_i^l (K_j^l)^T}{\sqrt{d_k}}. \quad (27)$$

The normalized interaction coefficient is

$$\alpha_{ij}^l = \frac{\exp(e_{ij}^l)}{\sum_{k=1}^N \exp(e_{ik}^l)}. \quad (28)$$

The particle interaction term is

$$m_i^l = \sum_{j=1}^N \alpha_{ij}^l V_j^l. \quad (29)$$

Thus,

$$M^l = A^l V^l. \quad (30)$$

8 Reaction-Diffusion Interpretation

MPNOT interprets the attention mechanism through a multi-particle reaction-diffusion framework.

A generic reaction-diffusion equation is

$$\frac{\partial u}{\partial t} = D \nabla^2 u + R(u). \quad (31)$$

The diffusion component describes spatial propagation,

$$D \nabla^2 u, \quad (32)$$

while the reaction component describes nonlinear local interaction,

$$R(u). \quad (33)$$

In the MPNOT interpretation, the attention mechanism provides the particle-interaction or reaction component.

The spatial propagation component is represented using a finite difference convolution operation.

9 Finite-Difference Convolution

Let u_i denote the field value at grid point i .

A discrete one-dimensional Laplacian can be written as

$$\nabla^2 u_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}. \quad (34)$$

In two dimensions,

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2}. \quad (35)$$

MPNOT incorporates a finite-difference convolution (FD-Conv) module to represent local spatial propagation.

Denote this operation by

$$\text{FDConv}(H^l). \quad (36)$$

10 MPNOT Layer

The MPNOT layer combines the multi-particle attention contribution with the finite-difference convolution.

Let

$$M^l = \text{MPAttention}(H^l). \quad (37)$$

Let

$$C^l = \text{FDConv}(H^l). \quad (38)$$

The layer can be represented schematically as

$$\boxed{H^{l+1} = H^l + M^l + C^l}. \quad (39)$$

A nonlinear transformation is then applied,

$$H^{l+1} = \sigma(H^l + M^l + C^l). \quad (40)$$

The reaction-like component is associated with the particle interaction, while the diffusion-like component is represented by FD-Conv.

11 Residual Transformer Block

A practical block can be written as

$$\tilde{H}^l = H^l + \text{MPAttention}\left(\text{LN}(H^l)\right), \quad (41)$$

followed by

$$H^{l+1} = \tilde{H}^l + \text{MLP}\left(\text{LN}(\tilde{H}^l)\right). \quad (42)$$

The MPNOT-specific spatial contribution can be incorporated as

$$H^{l+1} = \tilde{H}^l + \text{FDConv}\left(\tilde{H}^l\right) + \text{MLP}\left(\text{LN}(\tilde{H}^l)\right). \quad (43)$$

12 Input Lifting

The input field is lifted into a latent feature space.

For an input

$$a(x), \quad (44)$$

the initial representation is

$$h_0(x) = P_\theta(a(x), x). \quad (45)$$

For discretized points,

$$H_i^0 = P_\theta(a(x_i), x_i). \quad (46)$$

Thus

$$H^0 \in \mathbb{R}^{N \times d}. \quad (47)$$

13 MPNOT Operator

After L multi-particle layers,

$$H^L = \mathcal{M}_\theta^L \circ \dots \circ \mathcal{M}_\theta^1(H^0). \quad (48)$$

The output projection is

$$\hat{u}(x) = Q_\theta(H^L(x)). \quad (49)$$

The complete learned operator is therefore

$$\boxed{\mathcal{G}_\theta = Q_\theta \circ \mathcal{M}_\theta^L \circ \dots \circ \mathcal{M}_\theta^1 \circ P_\theta}. \quad (50)$$

14 Multi-Head Multi-Particle Attention

For H attention heads,

$$Q_h = HW_Q^h, \quad (51)$$

$$K_h = HW_K^h, \quad (52)$$

and

$$V_h = HW_V^h. \quad (53)$$

Each head computes

$$Z_h = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d_h}}\right) V_h. \quad (54)$$

The heads are concatenated:

$$Z = \text{Concat}(Z_1, \dots, Z_H). \quad (55)$$

A final projection gives

$$\text{MPAttention}(H) = ZW_O. \quad (56)$$

15 Training Objective

Given training data

$$\mathcal{D} = \left\{ (a^{(n)}, u^{(n)}) \right\}_{n=1}^{N_{\text{train}}}, \quad (57)$$

the predicted solution is

$$\hat{u}^{(n)} = \mathcal{G}_\theta \left(a^{(n)} \right). \quad (58)$$

The mean-squared-error objective is

$$\mathcal{L} = \frac{1}{N_{\text{train}}} \sum_{n=1}^{N_{\text{train}}} \left\| \hat{u}^{(n)} - u^{(n)} \right\|_2^2. \quad (59)$$

The relative L^2 error is

$$\mathcal{E}_{\text{rel}} = \frac{\|\hat{u} - u\|_2}{\|u\|_2}. \quad (60)$$

16 PyTorch Implementation

```
1
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6
7 class MultiParticleAttention(nn.Module):
8
9     def __init__(
10         self,
11         dim,
12         heads=4
13     ):
14         super().__init__()
15
16         assert dim % heads == 0
17
18         self.dim = dim
19         self.heads = heads
20         self.head_dim = dim // heads
21
22         self.q = nn.Linear(
23             dim,
24             dim
25         )
26
27         self.k = nn.Linear(
28             dim,
29             dim
30         )
31
32         self.v = nn.Linear(
33             dim,
34             dim
```

```

35         )
36
37         self.out = nn.Linear(
38             dim,
39             dim
40         )
41
42     def forward(self, x):
43
44         B, N, C = x.shape
45
46         q = self.q(x)
47         k = self.k(x)
48         v = self.v(x)
49
50         q = q.view(
51             B,
52             N,
53             self.heads,
54             self.head_dim
55         ).transpose(1, 2)
56
57         k = k.view(
58             B,
59             N,
60             self.heads,
61             self.head_dim
62         ).transpose(1, 2)
63
64         v = v.view(
65             B,
66             N,
67             self.heads,
68             self.head_dim
69         ).transpose(1, 2)
70
71         scores = torch.matmul(
72             q,
73             k.transpose(-2, -1)
74         )
75
76         scores = (
77             scores
78             / (self.head_dim ** 0.5)
79         )
80
81         attention = F.softmax(
82             scores,
83             dim=-1
84         )
85
86         interaction = torch.matmul(
87             attention,
88             v
89         )
90
91         interaction = interaction.transpose(
92             1,

```



```

93         2
94     ).contiguous()
95
96     interaction = interaction.view(
97         B,
98         N,
99         C
100    )
101
102    return self.out(
103        interaction
104    )
105
106
107    class FDConv2D(nn.Module):
108
109        def __init__(
110            self,
111            channels
112        ):
113            super().__init__()
114
115            self.conv = nn.Conv2d(
116                channels,
117                channels,
118                kernel_size=3,
119                padding=1
120            )
121
122        def forward(
123            self,
124            x,
125            height,
126            width
127        ):
128
129            B, N, C = x.shape
130
131            x = x.transpose(
132                1,
133                2
134            )
135
136            x = x.reshape(
137                B,
138                C,
139                height,
140                width
141            )
142
143            x = self.conv(x)
144
145            x = x.reshape(
146                B,
147                C,
148                N
149            )
150

```

```

151         x = x.transpose(
152             1,
153             2
154         )
155
156         return x
157
158
159 class MPNOTBlock(nn.Module):
160
161     def __init__(
162         self,
163         dim,
164         heads=4,
165         mlp_ratio=4
166     ):
167         super().__init__()
168
169         self.norm1 = nn.LayerNorm(
170             dim
171         )
172
173         self.attention = (
174             MultiParticleAttention(
175                 dim,
176                 heads
177             )
178         )
179
180         self.norm2 = nn.LayerNorm(
181             dim
182         )
183
184         self.mlp = nn.Sequential(
185             nn.Linear(
186                 dim,
187                 dim * mlp_ratio
188             ),
189             nn.GELU(),
190             nn.Linear(
191                 dim * mlp_ratio,
192                 dim
193             )
194         )
195
196     def forward(
197         self,
198         x
199     ):
200
201         x = (
202             x
203             +
204             self.attention(
205                 self.norm1(x)
206             )
207         )
208

```

```

209         x = (
210             x
211             +
212             self.mlp(
213                 self.norm2(x)
214             )
215         )
216
217         return x
218
219
220 class MPNOT(nn.Module):
221
222     def __init__(
223         self,
224         in_channels,
225         out_channels,
226         coord_dim=2,
227         width=64,
228         depth=4,
229         heads=4
230     ):
231         super().__init__()
232
233         self.input_projection = nn.Linear(
234             in_channels + coord_dim,
235             width
236         )
237
238         self.blocks = nn.ModuleList(
239             [
240                 MPNOTBlock(
241                     dim=width,
242                     heads=heads
243                 )
244                 for _ in range(depth)
245             ]
246         )
247
248         self.output_projection = nn.Sequential(
249             nn.Linear(
250                 width,
251                 width
252             ),
253             nn.GELU(),
254             nn.Linear(
255                 width,
256                 out_channels
257             )
258         )
259
260     def forward(
261         self,
262         coords,
263         x
264     ):
265
266         x = torch.cat(

```

```

267         [
268             coords,
269             x
270         ],
271         dim=-1
272     )
273
274     x = self.input_projection(
275         x
276     )
277
278     for block in self.blocks:
279
280         x = block(x)
281
282     return self.output_projection(
283         x
284     )
285
286
287 if __name__ == "__main__":
288
289     device = (
290         "cuda"
291         if torch.cuda.is_available()
292         else "cpu"
293     )
294
295     batch_size = 2
296     n_points = 256
297
298     coords = torch.rand(
299         batch_size,
300         n_points,
301         2,
302         device=device
303     )
304
305     field = torch.randn(
306         batch_size,
307         n_points,
308         1,
309         device=device
310     )
311
312     model = MPNOT(
313         in_channels=1,
314         out_channels=1,
315         coord_dim=2,
316         width=64,
317         depth=4,
318         heads=4
319     ).to(device)
320
321     prediction = model(
322         coords,
323         field
324     )

```

```

325
326     print(
327         "Input:",
328         field.shape
329     )
330
331     print(
332         "Prediction:",
333         prediction.shape
334     )

```

17 Training

```

1
2 optimizer = torch.optim.Adam(
3     model.parameters(),
4     lr=1e-3
5 )
6
7 criterion = nn.MSELoss()
8
9 for epoch in range(100):
10
11     model.train()
12
13     optimizer.zero_grad()
14
15     prediction = model(
16         train_coords,
17         train_input
18     )
19
20     loss = criterion(
21         prediction,
22         train_target
23     )
24
25     loss.backward()
26
27     optimizer.step()
28
29     if epoch % 10 == 0:
30
31         print(
32             f"Epoch {epoch}: "
33             f"Loss = {loss.item():.6e}"
34         )

```

18 Summary

MPNOT interprets Transformer attention as an interacting multi-particle system.

The central attention interaction is

$$\alpha_{ij} = \text{softmax}_j \left(\frac{Q_i K_j^T}{\sqrt{d_k}} \right), \quad (61)$$

and the corresponding particle interaction is

$$m_i = \sum_{j=1}^N \alpha_{ij} V_j. \quad (62)$$

The MPNOT architecture combines this multi-particle interaction with spatial finite-difference operations and nonlinear Transformer processing:

$$\boxed{H^{l+1} = \mathcal{M}_\theta \left(H^l \right)}. \quad (63)$$

The complete neural operator is

$$\boxed{\hat{u} = Q_\theta \circ \mathcal{M}_\theta^L \circ \cdots \circ \mathcal{M}_\theta^1 \circ P_\theta(a)}. \quad (64)$$

The essential conceptual correspondence is

$$\boxed{\text{Attention} \longleftrightarrow \text{Particle Interaction}} \quad (65)$$

and

$$\boxed{\text{FD-Conv} \longleftrightarrow \text{Spatial Diffusion}}. \quad (66)$$